



AUDIT REPORT

December 2025

For



Yodl

Table of Content

Executive Summary	04
Note	06
Number of Security Issues per Severity	10
Summary of Issues	11
Checked Vulnerabilities	13
Techniques and Methods	15
Types of Severity	17
Types of Issues	18
Severity Matrix	19
 Critical Severity Issues	20
1. Withdrawal Share Override Within Same Epoch Causes Loss of User Funds	20
 High Severity Issues	22
2. Front-Running Reward Distribution Allows Disproportionate Reward Capture	22
3. Wrong Epoch Settlement Enables Theft of Other Users' Withdrawals	24
4. Redelegation Cancellation Allows Front-Runner to Capture Full Epoch Settlement	26
 Medium Severity Issues	27
5. Cliff Lock Bypass Due to Missing Unlock-Time Validation	27
6. Single Global maxStaleness Used Across Multiple Chainlink Price Feeds With Different Heartbeats	28
7. Reward Distribution Broken Without Continuous Notifications	29
8. ETH Price Deviation Breaker Fails After Prolonged Inactivity	30
9. Redelegation Cancellation Incorrectly Resets Partially Withdrawn Positions	31



10. Inconsistent Redelegating State When Claiming Withdraw During Active Redelegating & Inconsistent Accounting After Partial Withdraw	32
11. Pending epochs can become unclaimable after redelegation vault switch	33
12. Repeated Deposits Can Permanently Prevent Reward Accrual Due to Baseline Debt Reset on Pending Share Activation	34
 Low Severity Issues	39
13. Missing Slippage Protection in External Vault Interactions	39
14. Incomplete Funding Adjustment Logic in Direct Deposit	40
 Informational Issues	41
15. Missing Two-Step Ownership Transfer	41
16. RewardsDistributor.sweep() can remove tokens still owed to users	42
17. Upgradeable init bug: _nextTokenId = 1 won't apply to proxy storage	43
Functional Tests	44
Automated Tests	46
Threat Model	47
Closing Summary & Disclaimer	52



Executive Summary

Project Name	Yodl
Protocol Type	Execution Coordination Layer
Project URL	https://www.yodl.fi
Overview	Yodl enables delegated execution through non-transferable NFT-based positions. Users lock YODL tokens into predefined delegation configurations and authorize whitelisted operators to access bounded execution capacity under protocol-enforced constraints. Four main contracts work together: Delegation (NFT positions), DelegationPassAccess(3 Passes with different multipliers), Rewards Distributor (fair reward allocation), and YODL Token (governance-enabled with entity-based minting). Key concerns include front-running risks in approval patterns, potential precision issues in reward calculations, and centralization via admin controls. The system uses Chainlink oracles for pricing and implements security features like deviation breakers, but requires careful auditing before deployment.
Audit Scope	The scope of this Audit was to analyze the Yodl Smart Contracts for quality, security, and correctness.
Source Code link	https://github.com/Everything-YODL/IOO/tree/main/contracts
Branch	main
Contracts in Scope	contracts/Delegation.sol contracts/DelegationPassAccess.sol contracts/YODL.sol contracts/RewardsDistributor.sol
Commit Hash	39524073dfed17ef6ac6d013bd5b5137fae814a0
Language	Solidity
Blockchain	Ethereum



Method	Manual Analysis, Functional Testing, Automated Testing
Review 1	3rd November 2025 - 18th November 2025
Updated Code Received	12th December 2025
Review 2	19th December 2025 - 29th December 2025
Fixed In	https://github.com/Everything-YODL/Delegation-Pass-Access/tree/7e3b1e322bdf4b1369e88e559c1fe31efa6e03b0

Verify the Authenticity of Report on QuillAudits Leaderboard:

<https://www.quillaudits.com/leaderboard>



Note

Yodl team on 6th feb 2026 pushed update which introduces a system-wide terminology(functions, contract names) refactor.

The terminology update corrects a legacy semantic mismatch where earlier names no longer accurately reflected the protocol's authorization-based execution and proceeds-accounting architecture. The refactor improves technical precision for auditors, integrators, and future extensibility, without altering any Business logic like execution, settlement, or accounting mechanics, or participant entitlements. Business logic remains same.

Below are listed terminology changes:

1. YODL Contract

Entity Renaming & Cap Reassignment

Updated the Entity enum and related mappings (entityCap, entityMinted) to reflect new allocation naming conventions:

- Operations → DevelopmentOps
- Contingency → ProtocolOps
- DAOTreasury → ProtocolReserve

These renamings are consistently applied across:

- Entity cap configuration
- Mint tracking logic
- Emitted events related to entity caps

Cleanup

Removed an obsolete comment referring to a "Delegation Pass cap", which is no longer relevant.

2. RewardsDistributor Contract→ AuthorizerProceedsRouter

Contract & Interface Renaming

- RewardsDistributor renamed to AuthorizerProceedsRouter
- IDelegation interface renamed to IAuthorization
- All references to delegation replaced with authorization

Token & Proceeds Tracking

- rewardTokens → proceedsTokens
- everRewardTokens → everProceedsTokens



State Variable Renaming

- totalActiveShares → totalActiveUnits
- accRPS → accProceedsPerUnit
- sharesByTokenId → unitsByTokenId
- rewardDebt → proceedsDebt
- unclaimed → unappliedProceeds
- pendingSharesByTokenId → pendingUnitsByTokenId

Event Renaming

- RewardTokenAdded → ProceedsTokenAdded
- SharesIncreased → UnitsIncreased

Function Renaming

- notifyReward → recordProceeds
- onSharesIncrease → onUnitsIncrease
- harvest → applyProceeds
- harvestAll → applyProceedsAll

Internal Logic Alignment

- Helper functions updated for consistency:
 - _consumePending → _usePending
 - _consumeActiveAndRebaseline → _useActiveRebase

Distribution logic remains functionally equivalent, but now consistently uses proceeds and units terminology.

Terminology Consistency

- All references to “rewards” replaced with “proceeds” or “fees”
- All references to “shares” replaced with “units”
- Example: “warm-up delay for new shares” → “warm-up delay for new units”

3. Delegation Pass Sale Contract → Authorization Pass Sale

Contract & Interface Renaming

- DelegationPassAccess → AuthorizationPassAccess
- IDelegation replaced with IAuthorization

Terminology Updates

- “Deposit and delegate” → “Deposit and authorize”
- “Buyer” → “Beneficiary”
- “Pass purchased” events → “Authorization granted”
- “Tokens” → “Units”
- “Cliff duration” → “Lock duration”
- “Purchased tokens / tokens sold” → “Units by address / units issued”
- “Min token purchase” → “Min token issuance”

Sale Lifecycle Renaming

- saleFinalized → windowClosed
- finalizeSale → closeWindow
- SaleNotPaused → WindowNotPaused



Logic Adjustments

- Maximum allocation calculations now operate on units instead of tokens
- Contract interaction updated to use IAuthorization

Naming updates:

- hardCapTokens (formerly hard cap)
- totalAuthorizedTokens (formerly total sold tokens)

Function Renaming

- buyWithEth → payEth
- buyWithStable → payStable
- quoteBuyWithEth → quoteEth

Configuration Updates

- basePriceUsdPerToken → baseUsd
- PassConfig.priceMultiplierBps → priceBps

4. Delegation Contract → Authorization

Contract & Interface Renaming

- Delegation renamed to Authorization
- IRewardsDistributor replaced with IAuthorizerProceedsRouter

Core Data Structure Changes

- Position struct renamed to Authorize
- All mappings and logic updated from positions to authorizations
- Internal accounting updated from shares → units

Event & Error Renaming

- WithdrawRequested → ExitRequested
- WithdrawClaimed → Exited
- RedelegationInitiated → ReauthorizationInitiated
- RedelegationFinalized → ReauthorizationFinalized
- RedelegationTargetCleared → ReauthorizationTargetCleared
- RedelegationCancelled → ReauthorizationCancelled
- HasRedelegationShares → HasReauthorizationShares
- WithdrawLocked → ExitLocked
- RewardsNotSet → AuthorizerProceedsRouterNotSet

Method Renaming

- depositAndDelegate → depositAndAuthorize
- requestWithdraw → requestExit
- initiateRedelegation → initiateReauthorization
- finalizeRedelegation → finalizeReauthorization
- Internal Logic & Storage Updates
- _mintPosition → _mintAuth
- redelegateTarget → reauthorizeTarget



Epoch settlement logic updated to use exit and reauthorize terminology:

- `_claimAndSplit`
- `_settleEpoch`
- `_clearPendingEpochs`

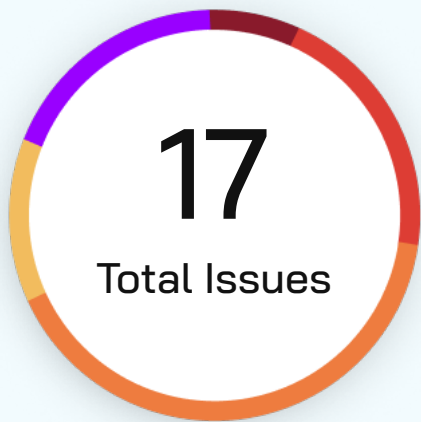
Funding Logic Update

- `_applyFundingOnDirectDeposit` → `_applyChargeDD`, indicating revised accounting semantics

Initialization Update

- ERC721 initialization updated:
 - Name: "Authorization Pass"
 - Symbol: "A-Pass"
 - (Previously "Delegation Pass" / "D-Pass")

Number of Issues per Severity



Critical	1 (5.88%)
High	3 (17.64%)
Medium	7 (41.17%)
Low	2 (11.76%)
Informational	3 (17.64%)

		Severity				
		Critical	High	Medium	Low	Informational
Issues	Open	0	0	0	0	0
	Acknowledged	0	0	2	2	0
	Partially Resolved	0	0	0	0	0
	Resolved	1	3	5	0	3



Summary of Issues

Issue No.	Issue Title	Severity	Status
1	Withdrawal Share Override Within Same Epoch Causes Loss of User Funds	Critical	Resolved
2	Front-Running Reward Distribution Allows Disproportionate Reward Capture	High	Resolved
3	Wrong Epoch Settlement Enables Theft of Other Users' Withdrawals	High	Resolved
4	Redelegation Cancellation Allows Front-Runner to Capture Full Epoch Settlement	High	Resolved
5	Cliff Lock Bypass Due to Missing Unlock-Time Validation	Medium	Resolved
6	Single Global maxStaleness Used Across Multiple Chainlink Price Feeds With Different Heartbeats	Medium	Resolved
7	Reward Distribution Broken Without Continuous Notifications	Medium	Acknowledged
8	ETH Price Deviation Breaker Fails After Prolonged Inactivity	Medium	Acknowledged
9	Redelegation Cancellation Incorrectly Resets Partially Withdrawn Positions	Medium	Resolved



Issue No.	Issue Title	Severity	Status
10	Inconsistent Redelegation State When Claiming Withdraw During Active Redelegation & Inconsistent Accounting After Partial Withdraw	Medium	Resolved
11	Pending epochs can become unclaimable after redelegation vault switch	Medium	Resolved
12	Repeated Deposits Can Permanently Prevent Reward Accrual Due to Baseline Debt Reset on Pending Share Activation	Medium	Resolved
13	Missing Slippage Protection in External Vault Interactions	Low	Acknowledged
14	Incomplete Funding Adjustment Logic in Direct Deposit	Low	Acknowledged
15	Missing Two-Step Ownership Transfer	Informational	Resolved
16	RewardsDistributor.sweep() can remove tokens still owed to users	Informational	Resolved
17	Upgradeable init bug: _nextTokenId = 1 won't apply to proxy storage	Informational	Resolved



Checked Vulnerabilities

✓ Access Management

✓ Arbitrary write to storage

✓ Centralization of control

✓ Ether theft

✓ Improper or missing events

✓ Logical issues and flaws

✓ Arithmetic Computations
Correctness

✓ Race conditions/front running

✓ SWC Registry

✓ Re-entrancy

✓ Timestamp Dependence

✓ Gas Limit and Loops

✓ Exception Disorder

✓ Gasless Send

✓ Use of tx.origin

✓ Malicious libraries

✓ Compiler version not fixed

✓ Address hardcoded

✓ Divide before multiply

✓ Integer overflow/underflow

✓ ERC's conformance

✓ Dangerous strict equalities

✓ Tautology or contradiction

✓ Return values of low-level calls



✓ **Missing Zero Address Validation**

✓ **Private modifier**

✓ **Revert/require functions**

✓ **Multiple Sends**

✓ **Using suicide**

✓ **Using delegatecall**

✓ **Upgradeable safety**

✓ **Using throw**

✓ **Using inline assembly**

✓ **Style guide violation**

✓ **Unsafe type inference**

✓ **Implicit visibility level**



Techniques and Methods

Throughout the audit of smart contracts, care was taken to ensure:

- The overall quality of code
- Use of best practices
- Code documentation and comments, match logic and expected behavior
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper
- Implementation of ERC standards
- Efficient use of gas
- Code is safe from re-entrancy and other vulnerabilities

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.



Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

Tools and Platforms Used for Audit

Remix IDE, Foundry, Solhint, Mythril, Slither, Solidity Static Analysis.



Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

■ **Critical: Immediate and Catastrophic Impact**

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

■ **High (H): Significant Risk of Major Loss or Compromise**

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

■ **Medium (M): Potential for Moderate Harm Under Specific Circumstances**

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

■ **Low (L): Minor Imperfections with Limited Repercussions**

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

■ **Informational (I): Opportunities for Improvement, Not Immediate Risks**

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.



Types of Issues

Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

Resolved

These are the issues identified in the initial audit and have been successfully fixed.

Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

		Impact		
		High	Medium	Low
Likelihood	High	Critical	High	Medium
	Medium	High	Medium	Low
	Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely.
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.



Critical Severity Issues

Withdrawal Share Override Within Same Epoch Causes Loss of User Funds

Resolved

Path

Delegation.sol

Function Name

`_addPendingEpoch`

Description

The `_addPendingEpoch()` function does not accumulate shares when multiple withdrawal requests for the same `tokenId` occur within the same epoch. Instead, it overwrites the previous amount stored in `epochShares[tokenId][epochId]`.

```
1 function _addPendingEpoch(uint256 tokenId, uint256 epochId, uint256 shares) internal {
2     pendingEpochs[tokenId].push(epochId);
3     epochIndex[tokenId][epochId] = pendingEpochs[tokenId].length; // index+1
4     epochShares[tokenId][epochId] = shares; // @audit
5 }
```

For example, if a user withdraws 100 shares in epoch 5 and later withdraws an additional 50 shares in the same epoch, the mapping will store only 50 instead of 150. Although `pending[tokenId].shares` correctly tracks the total pending withdrawal shares, the epoch-specific tracking becomes inaccurate. This inconsistency causes incorrect claim amounts, failed accounting during claim or redelegation flows, and makes the system unable to reconcile proportional returns for a given epoch. As a result, users may receive less (or more) than they are owed, and the Delegation contract's internal accounting diverges from the actual withdrawn share state.

Impact - High

The overwritten epoch share values cause the contract to lose track of how many shares were truly withdrawn in that epoch, leading to incorrect claim amounts and potential loss or misallocation of user funds during withdrawal or redelegation settlement.

Likelihood

High



POC

- User requests a withdrawal of 100 shares in epoch 7.
- `_addPendingEpoch()` stores `epochShares[tokenId][7] = 100`.
- User later withdraws another 50 shares in the same epoch 7.
- `_addPendingEpoch()` overwrites the stored value to 50 instead of increasing to 150.
- When claiming, the contract only processes the last recorded 50 shares but still considers 150 shares in the global pending counter.
- This mismatch produces incorrect claim results and breaks expected withdrawal accounting.

Recommendation

The contract should maintain accurate per-epoch share accounting by ensuring that multiple withdrawals within the same epoch are aggregated rather than replaced

High Severity Issues

Front-Running Reward Distribution Allows Disproportionate Reward Capture

Resolved

Path

RewardsDistributor.sol

Function

`notifyReward()`

Description

The `RewardsDistributor.notifyReward()` function allocates rewards based solely on the current total shares (`totalActiveShares`) at the moment of notification. Since there is no mechanism to enforce a staking duration or epoch boundary, an attacker can deposit a large number of shares immediately before a reward is distributed and capture a disproportionate amount of the reward even without prior participation.

Impact

Attackers can repeatedly front-run reward distributions to drain rewards intended for long-term participants. Honest users receive significantly reduced rewards, and the overall fairness and integrity of the reward mechanism are compromised.

Likelihood

High

POC

```
function test_FrontRunRewardDistribution() public {
    uint256 depositAmountUser1 = 1 ether;
    uint256 depositAmountUser2 = 1000 ether;

    console.log("=== Initial Balances ===");
    console.log("User1:", yodl.balanceOf(user1));
    console.log("User2:", yodl.balanceOf(user2));

    // User1 deposits early
    vm.startPrank(user1);
    (uint256 tokenId1, ) = delegation.directDeposit(user1, operator1,
depositAmountUser1);
    vm.stopPrank();

    // Attacker deposits just before reward
    vm.startPrank(user2);
    (uint256 tokenId2, ) = delegation.directDeposit(user2, operator1,
depositAmountUser2);
    vm.stopPrank();
}
```



```
uint256 beforeRewardU1 = yodl.balanceOf(user1);
uint256 beforeRewardU2 = yodl.balanceOf(user2);

// Operator adds reward
vm.startPrank(operator1);
rewardsDistributor.notifyReward(operator1, address(yodl), 1000 ether);
vm.stopPrank();

// Harvest - FIXED: Use helper function to create array
vm.prank(user1);
rewardsDistributor.harvest(tokenId1, _toSingleToken(address(yodl)));
vm.prank(user2);
rewardsDistributor.harvest(tokenId2, _toSingleToken(address(yodl)));

uint256 afterU1 = yodl.balanceOf(user1);
uint256 afterU2 = yodl.balanceOf(user2);

uint256 rewardU1 = afterU1 - beforeRewardU1;
uint256 rewardU2 = afterU2 - beforeRewardU2;

console.log("User1 reward:", rewardU1);
console.log("User2 reward:", rewardU2);

// Assert front-runner dominates reward capture
assertGt(rewardU2, rewardU1 * 100);
}
```

Recommendation

Use a two-step deposit flow (`processDeposit` → `confirmDeposit`) so new deposits become reward-eligible only after confirmation, preventing same-block front-running.



Wrong Epoch Settlement Enables Theft of Other Users' Withdrawals

Resolved

Path

Delegation.sol

Function

`requestWithdraw()`

Description

The Delegation contract records user withdrawals using the epochId returned by `ISymbioticVault.currentEpoch()`. However, the underlying Symbiotic vault performs withdrawal settlement at the epoch level, not the user level. As a result, all users who withdraw during the same epoch are implicitly grouped into a shared withdrawal pool, even though Delegation internally treats them as separate claimable entries.

When a user calls `requestWithdraw()`, the Delegation contract:

1. Obtains the vault's current global epoch.
2. Calls `vault.withdraw(address(this), shares)`.
3. Stores the user's pending withdrawal under that epochId.

If multiple users withdraw in the same epoch, all of their withdrawal requests share the same epochId, and the vault combines them into one undifferentiated epoch withdrawal bucket.

Later, when any user claims their withdrawal via `claimWithdraw()`, the Delegation contract calls:

`vault.claim(address(this), epochId)`

This call returns the entire payout for that epoch, which may include underlying that belongs to several different users. Delegation then transfers this entire amount to the caller, even if they only contributed a fraction of the epoch's total withdrawn shares.

Because the vault does not return user-specific claim amounts, the Delegation contract cannot enforce proportional distribution. The first user who calls `claimWithdraw()` for that epoch receives all the epoch's settled underlying, while other users with pending withdrawals from the same epoch are left with nothing.

This creates a direct path for loss of funds and breaks user isolation guaranteed by NFTs representing positions.

Impact - High

Users can lose their withdrawn underlying to other users. A malicious or fast user can intentionally withdraw small shares early in an epoch, wait for others to request withdrawals, and then claim the entire epoch payout once settlement becomes available.

Likelihood

Medium



POC

- User A withdraws 100 shares during epoch 12.
- User B withdraws 1,000 shares during the same epoch 12.
- Both users receive an entry under epochId = 12 in pendingEpochs.
- Once epoch 12 is settled, User A calls claimWithdraw() first.
- vault.claim() returns the entire epoch payout (for both A and B).
Delegation transfers this entire amount to User A.
- User B later tries to claim epoch 12 but receives nothing, as the vault has no remaining funds for that epoch.

Recommendation

To ensure correct and isolated accounting for each user's withdrawal.



Redelegation Cancellation Allows Front-Runner to Capture Full Epoch Settlement

Resolved

Path

Delegation.sol

Function

`cancelRedelegation()`

Description

The `cancelRedelegation()` function processes pending epochs and redeposits the underlying back into the current vault. The issue arises because multiple users in the same vault epoch depend on the same `ISymbioticVault.claim()` settlement. If two users are waiting on the same epoch, whoever calls `cancelRedelegation()` first will receive the entire epoch settlement returned by `vault.claim()`. Since the vault does not track user-specific claim amounts, the next user calling the same function will receive nothing. This creates a front-running vulnerability where a user can intentionally monitor mempool activity and race to execute `cancelRedelegation()` before another redelegating user, capturing the entire claimable amount and causing the second user to lose their settlement completely.

Impact - High

A user can front-run another user's cancellation and receive the full epoch payout, leading to misallocation of funds and direct financial loss for users sharing the same pending epoch.

Likelihood

Medium

POC

- Two users, A and B, have pending shares in the same epoch due to redelegation.
- The epoch matures, meaning `vault.claim()` will now return underlying.
- User A sees a mempool transaction from User B calling `cancelRedelegation()`.
- User A sends a higher-gas transaction to front-run the call.
- User A receives 100% of the epoch's claimable underlying and redeposits it.
- User B calls the function afterward but receives nothing because the vault's epoch payout was already consumed by the earlier call.

Recommendation

The redelegation cancellation flow should ensure that each user's pending withdrawals are isolated from other users sharing the same epoch.



Medium Severity Issues

Cliff Lock Bypass Due to Missing Unlock-Time Validation

Resolved**Path**

Delegation.sol

Function Name`initiateRedelegation`**Description**

The contract enforces a lock period using `unlockTimestamp` to prevent withdrawals before the cliff ends; however, `initiateRedelegation()` does not validate whether the position is still locked, allowing the lock to be bypassed.

Impact - Medium

This flaw allows users to circumvent the lockup rules and prematurely withdraw or redelegate shares, enabling early access to funds and breaking the economic designs that depend on enforced locking

Likelihood

Medium

Recommendation

The redelegation process should enforce the same lock-time restrictions as the standard withdrawal path.



Single Global maxStaleness Used Across Multiple Chainlink Price Feeds With Different Heartbeats

Resolved

Description

The contract applies a single global maxStaleness value to validate freshness for all Chainlink price feeds. However, different Chainlink feeds are configured with different heartbeat (update interval) parameters. Using one universal staleness threshold can result in either accepting stale prices or rejecting valid prices, depending on the feed.

For example, the ETH/USD feed typically updates much more frequently (e.g., ~30 minutes heartbeat) compared to stablecoin feeds such as USDT/USD, which may have a heartbeat of up to 24 hours. A single maxStaleness value cannot safely accommodate both cases. You can see all the data feed and heart beat [here](#)

Root Cause

Chainlink price feeds are configured with feed-specific trigger parameters:

- Heartbeat: the maximum time allowed between updates, even if price deviation is small.
- Deviation threshold: the price movement required to trigger an update earlier.

Highly volatile assets such as ETH require frequent updates to maintain accuracy, hence a shorter heartbeat. Stablecoins, by design, experience low volatility and therefore use much longer heartbeats to reduce unnecessary updates.

The contract enforces staleness with a single global variable:

```
function setMaxStaleness(uint256 maxStalenessSeconds_) external
onlyRole(DEFAULT_ADMIN_ROLE) {
    maxStaleness = maxStalenessSeconds_;
    emit MaxStalenessUpdated(maxStalenessSeconds_);
}
```

This value is applied uniformly in _readUsdPrice:

```
if (maxStaleness > 0 && block.timestamp - updatedAt > maxStaleness) {
    revert FeedStaleOrInvalid(address(feed));
}
```

Because different feeds have different heartbeat guarantees:

- If maxStaleness is too long, the system may accept prices that Chainlink itself considers stale for fast-updating feeds (e.g., ETH/USD).
- If maxStaleness is too short, the system may reject valid data for slow-updating feeds (e.g., USDT/USD), causing unnecessary reverts and potential protocol disruption.

Recommendation

Use feed-specific staleness thresholds instead of a single global maxStaleness. [Reference](#)



Reward Distribution Broken Without Continuous Notifications

Acknowledged

Path

RewardsDistributor.sol

Function Name

`notifyReward()`

Description

The RewardsDistributor contract uses an accRPS (Accumulated Rewards Per Share) mechanism that requires continuous reward notifications to function. If `notifyReward()` is called only once (or stops being called), all users who join after the initial distribution will have identical debt values, making their claimable rewards permanently zero, regardless of how long they stake.

This fundamentally breaks the reward distribution system and results in:

1. Permanent fund lock - Rewards stuck in contract forever
2. User fund loss - Stakers receive zero rewards despite correct staking
3. Systemic failure - Protocol becomes non-functional without continuous distributions
4. Economic irrationality - Reward distributors lose funds with no benefit to users

Impact

Medium

Likelihood

Medium

Recommendation

Implement time-weighted rewards before mainnet deployment. This is not a simple patch, it requires rethinking the reward distribution architecture to be time-based rather than event-based.



ETH Price Deviation Breaker Fails After Prolonged Inactivity

Acknowledged

Path

IOOCrowdsale.sol

Function Name

`_snapEthPrice()`

Description

The contract implements an ETH/USD deviation-breaker that rejects oracle prices deviating too far from the last accepted price.

However, `lastEthPrice1e18` is updated only when a successful ETH purchase occurs, meaning:

- If no ETH purchases occur for days/weeks, `lastEthPrice1e18` becomes stale.
- When a user eventually tries to buy with ETH again, the new oracle price—although fresh—may differ by more than `maxDeviationBps` from the stale cached price.
- As a result, `_snapEthPrice()` always reverts, making `buyWithEth()` and ETH quoting permanently unusable until governance manually unblocks it.

This becomes a self-inflicted circuit break that does not reflect real oracle conditions, causing denial-of-service.

Impact

Medium

Likelihood

Medium

Recommendation

Implement a mechanism where `lastEthPrice1e18` should not be outdated and if it becomes outdated then `maxDeviationBps` should be updated so that operations doesn't stop.



Redelegation Cancellation Incorrectly Resets Partially Withdrawn Positions

Resolved

Path

Delegation.sol

Function Name

`cancelRedelegation()`

Description

The contract allows a position to have part of its shares pending withdrawal while the remaining shares are actively redelegated. However, when `cancelRedelegation()` is executed, the function treats the cancellation as if all remaining active shares belong to the redelegation process, even if only part of the position originally entered redelegation. This becomes problematic when a user performs a partial withdrawal and then immediately initiates redelegation with the leftover shares.

For example, if a tokenId holds 100 shares, the user may request withdrawal for 50 shares, leaving 50 shares as active. If the user then initiates redelegation for the remaining 50 active shares, calling `cancelRedelegation()` later causes the entire redelegation flow to execute without considering that only part of the position entered redelegation. As soon as `cancelRedelegation()` is triggered, the contract processes all pending epochs associated with redelegation and then restores shares back to the main position – effectively resetting the entire share distribution. This behavior collapses the intended partial state split (50 pending withdrawal, 50 active) back into a single unified active state, overriding the user's intended action sequence.

Impact - Medium

A user's position can unintentionally merge or reset when partial withdrawals and redelegation flows overlap, causing unexpected state transitions and potentially allowing users to bypass intended sequencing of withdrawals and redelegations.

POC

- A position has 100 total shares.
- The user requests withdrawal of 50 shares → 50 become pending, 50 remain active.
- The user immediately initiates redelegation for the remaining 50 shares.
Later, the user calls `cancelRedelegation()`.
- Instead of canceling only the redelegated 50 shares, the cancellation flow consolidates the entire state back into a single active share balance.
- The user's partial withdrawal structure is unintentionally overridden, leading to inconsistent and unexpected position behavior.

Recommendation

Redelegation cancellation should only operate on the exact subset of shares that were part of the redelegation process and must not interfere with previously requested withdrawals.



Inconsistent Redelegation State When Claiming Withdraw During Active Redelegation & Inconsistent Accounting After Partial Withdraw

Resolved

Path

Delegation.sol

Function Name

`requestWithdraw(), claimWithdraw(), initiateRedelegation(), finalizeRedelegation()`

Description

The contract does not correctly separate pending-withdraw shares from active redelegation shares. When a user has partially withdrawn some of their shares and then initiates redelegation on the remaining shares, the internal accounting no longer reflects two distinct states. If the user then calls `claimWithdraw()` while redelegation is still active, the contract processes claims that belong to pending-withdraw shares while the position is simultaneously being manipulated through redelegation. Because both flows operate on the same pending and epochShares structures, this causes the contract to mix accounting for withdrawal operations and redelegation operations.

Similarly, if redelegation is initiated after a partial withdrawal, the redelegation logic assumes the full remaining share balance belongs entirely to the redelegation process. Any later claim or cancellation step then resets or merges the position state, ignoring the fact that only part of the position was originally withdrawn. As a result, the user's active shares, pending shares, and redelegated shares can become merged or overwritten, leading to an inconsistent and unpredictable position state.

Impact - Medium

Overlapping withdrawal and redelegation operations can corrupt a position's accounting, causing share balances to merge incorrectly, pending withdrawals to be overwritten, or redelegated shares to be misrepresented. This exposes users to unintended state resets, incorrect claim outcomes, and loss of expected share separation.

Likelihood - Medium

POC

- A position has 100 shares.
- The user requests withdrawal for 40 shares → now 40 pending, 60 active.
- The user initiates redelegation for the remaining 60 active shares.
Before redelegation is finalized, the user calls `claimWithdraw()` for one of the pending epochs.
- The claim updates pending.shares while redelegation still expects the original pending share count.
- Redelegation then finalizes or cancels and resets the position, collapsing the pending-active separation and causing inconsistent share accounting.

Recommendation

The contract should enforce a clear separation between withdrawal and redelegation states to prevent overlapping operations from corrupting accounting.

Auditor's Comment

In `reassignPendingShares()`, `redelegateTarget` should be cleared when all shares are converted.



Pending epochs can become unclaimable after redelegation vault switch

Resolved

Path

Delegation.sol

Function Name

`finalizeRedelegation()`

Description

Internal settlement uses `positions[tokenId].vault` to decide which vault to settle against via `_claimAndSplit()`.

If `finalizeRedelegation()` updates `positions[tokenId].vault` while there are still pending epochs created under the previous vault, subsequent claim operations will attempt to settle those epochs against the new vault, causing the settlement to fail.

POC

- User has a position in vault A.
- User calls `requestWithdraw()` (creates pending withdraw epoch e1 in vault A).
- Before claiming e1, user calls `initiateRedelegation()` to move remaining shares to operator/vault B (creates a pending redelegation epoch e2 in vault A).
- User later calls `finalizeRedelegation(tokenId, [e2])` while e1 is still pending.
- `finalizeRedelegation` settles e2 and updates `positions[tokenId].vault = vaultB` even though e1 remains pending from vault A.
- Now the user tries `claimWithdraw(tokenId, [e1])`. `claimWithdraw` uses `p.vault` (now vaultB) and `_claimAndSplit` uses `positions[tokenId].vault` (vaultB) → epoch accounting keys don't exist (`epochTotalShares` was stored under vaultA), so settlement reverts with `UnknownEpoch()`.
- Result: e1 is effectively stuck unless you upgrade the contract and add a recovery path (and the user knows the old vault).

Impact

Medium

Likelihood

Medium

Recommendation

Ensure that `finalizeRedelegation` does not update the position's vault while there are unresolved epochs from the previous vault. This can be enforced by requiring all pending epochs to be fully settled before finalizing redelegation, or by explicitly tracking the vault per epoch. Alternatively, disallow mixing pending withdrawals with redelegation to avoid cross-vault settlement issues.



Repeated Deposits Can Permanently Prevent Reward Accrual Due to Baseline Debt Reset on Pending Share Activation

Resolved

Path

Delegation.sol

Function Name

`finalizeRedelegation()`

Description

A user who repeatedly deposits after each warm-up period may never accumulate claimable rewards, despite holding active shares. This occurs because pending shares are activated and then immediately included in the baseline reward debt calculation, effectively resetting accrued rewards to zero on every cycle.

In contrast, if a user deposits again before the warm-up period elapses and harvests in between, rewards become claimable. This creates inconsistent behavior where reward eligibility depends on deposit timing rather than economic exposure.

Root Cause

The issue stems from the interaction between:

`_activateIfReady``_harvestToCredit``_baselineDebt`

when `onSharesIncrease` is called

1. Pending shares mature into active shares
2. `_baselineDebt` is recalculated using all active shares, including newly activated ones
3. Reward harvesting is done after activation, but baseline debt is updated before reward is claimed for the current active shares.
- 4.

```
// onSharesIncrease
```

```
_activateIfReady(tokenId, operator, p);
```

```
_harvestToCredit(tokenId, p);
```

```
// later...
```

```
_activateIfReady(tokenId, operator, p);
```

When pending shares are activated inside `_activateIfReady`, `_baselineDebt` is called with `newShares`:

```
rewardDebt[tokenId][t] =
```

```
    Math.mulDiv(newShares, p.accRPS[t], 1e18);
```

This sets the user's reward debt to the current accumulated rewards for all shares, including shares that were just activated and have not had time to earn rewards. As a result: when `_harvestToCredit` is called,

accrued rewards - baseline debt = 0



Proof of Concept

Run this test using foundry. The file is in the link here -

<https://github.com/OxSlowbug/Brokenreward.git>

It has 2 Scenarios. The reason they both have 1000 unclaimed after first deposit is because of the `releaseBufferOnFirstActivation` subsequently, the reward is improperly tracked.

Scenario 1 – Rewards Lost Due to Debt Reset (Broken Path)

Initial State:

Active shares: 0

Pending shares: 0

Reward debt: 0

Unclaimed: 0

Step-by-Step Execution

1. Step 1-2: Initial deposit + Notify reward

- Deposit 100 shares → added to pendingShares
- Notify 1000 reward tokens
- Warm-up timer set
- State: Active: 0, Pending: 100, Debt: 0, Unclaimed: 0

2. Step 3-5: Forward time beyond warmup + Deposit + Notify

- Time warped past warmup period
- Deposit 100 shares AFTER warmup completes
- Critical: Pending shares (100) move to active
- `_baselineDebt` recalculated using full active shares
- Notify 1000 more reward tokens
- Result at Step 6: Pending: 2000, Unclaimed: 1000
- ❌ Expected 2000 unclaimed, got only 1000 – 1000 tokens lost!

3. Step 7-9: Repeat – Forward time + Deposit + Notify

- Time warped past warmup period again
- Deposit 100 shares AFTER warmup completes
- Activation occurs → baseline debt reset again
- Notify 1000 more reward tokens
- Result at Step 10: Pending: 2000, Unclaimed: 1000
- ❌ Still only 1000 unclaimed – another 2000 tokens lost!

4. Step 11-13: Repeat again – Forward time + Deposit + Notify

- Time warped past warmup period
- Deposit 100 shares AFTER warmup completes
- Activation occurs → baseline debt reset yet again
- Notify 1000 more reward tokens
- Result at Step 14: Pending: 1999, Unclaimed: 1000
- ❌ Still only 1000 unclaimed – pattern confirms bug!

Root Cause

Each time a deposit occurs after warmup, it triggers activation which recalculates `_baselineDebt` using the current `accruedPerShare * activeShares`. This effectively “resets” the debt to match all accrued rewards, making `accrued - debt ≈ 0`, thereby erasing previously earned rewards.



Total Loss

- Total notified: 4000 tokens (4 × 1000)
- Total claimable: 1000 tokens
- Lost: 3000 tokens (75% of rewards)

Scenario 2 – Rewards Preserved (Working Path)

Initial State:

Active shares: 0

Pending shares: 0

Reward debt: 0

Unclaimed: 0

Step-by-Step Execution:

1. Step 1-3: Initial deposit + Notify + Forward time

- Deposit 100 shares → added to pendingShares
- Notify 1000 reward tokens
- Time warped past warmup period
- State: Active: 0, Pending: 100, Debt: 0, Unclaimed: 0

2. Step 4-5: Deposit AFTER warmup + Notify

- Deposit 100 shares AFTER warmup completes
- Pending shares (100) activate normally
- Notify 1000 reward tokens
- State: Active: 100, Pending: 100, Debt: 0, Unclaimed: 1000
- Result at Step 6: Pending: 2000, Unclaimed: 1000
-  1000 tokens properly accrued

3. Step 7-9: KEY DIFFERENCE – NO time forwarding + Deposit + Notify

- Step 7: Do NOT forward time (stay BEFORE next warmup)
- Step 8: Deposit 100 shares BEFORE warmup completes
- Critical: No activation occurs!
- _harvestToCredit runs using existing active shares
- Rewards properly accrue to unclaimed
- Notify 1000 more reward tokens
- Result at Step 10: Pending: 3000, Unclaimed: 2000
-  All rewards preserved! Debt correctly tracked at 1000

Impact

Users who regularly deposit after warm-up can be permanently excluded from rewards

Users who have pending rewards and deposit or withdraw after their warm up period expires loses that part of their reward for that session.

Recommendation

Reorder the operation

```
function onSharesIncrease(...) {
  require(deltaShares > 0, "RewardsDistributor: zero delta");
  Pool storage p = pool[operator];

-  _activateIfReady(tokenId, operator, p);
  _harvestToCredit(tokenId, p);
+  _activateIfReady(tokenId, operator, p);

  pendingSharesByTokenId[tokenId][operator] += deltaShares;
  // ... rest of function
}
```



Low Severity Issues

Missing Slippage Protection in External Vault Interactions

Acknowledged

Description

Functions in Delegation.sol such as depositAndDelegate, directDeposit, requestWithdraw, and claimWithdraw interact with external vaults (ISymbioticVault.deposit, withdraw, and claim) without enforcing any slippage bounds.

Since ISymbioticVault functions similarly to an ERC-4626 vault, changes in vault state (e.g., share price, fees, epoch rollovers, or liquidity changes) between transaction submission and execution may cause users to receive fewer shares on deposit or less underlying on withdrawal or claim than expected.

Impact

Users may suffer value loss due to price movement, vault parameter adjustments, or MEV-induced front-running between transaction broadcast and execution.

Likelihood

Medium

Recommendation

Introduce user-supplied minimum output checks (e.g., minSharesOut, minAmountOut) for all vault interactions.



Incomplete Funding Adjustment Logic in Direct Deposit

Acknowledged

Path

Delegation.sol

Function Name

`directDeposit`

Description

The `directDeposit()` function applies `_applyFundingOnDirectDeposit()` before processing the user's deposit. However, the funding adjustment logic is only a placeholder and currently returns the original amount untouched. This creates incomplete or misleading behavior, because the system signals that funding adjustments exist but never actually modifies the deposit amount. Any downstream components relying on accurate funding logic may behave incorrectly once the actual logic is added or when external expectations assume dynamic funding effects.

Impact

Users may suffer value loss due to price movement, vault parameter adjustments, or MEV-induced front-running between transaction broadcast and execution.

Likelihood

Low

Recommendation

This ensures that contract consumers do not assume funding adjustments are active when they are not.

Auditor's Comment

Any issues caused by changes in the `_applyFundingOnDirectDeposit` function are considered out of the audit scope.

Informational Issues

Missing Two-Step Ownership Transfer

Resolved

Path

Delegation.sol / RewardsDistributor.sol

Description

The contracts rely on OwnableUpgradeable from OpenZeppelin, which provides a single-step ownership transfer mechanism. This pattern immediately transfers ownership to the new address when `transferOwnership()` is called. If the new owner address is incorrect, mis-typed, compromised, or incapable of executing privileged operations, the system may become permanently locked or controlled by an unintended party.

Modern best practice is to use Two-Step Ownable, where ownership transfer occurs in two phases—`transferOwnership()` (proposes) and `acceptOwnership()` (accepts). This prevents accidental transfers, mitigates operational mistakes, and protects the protocol from misconfigured admin addresses.

Impact - Low

Incorrect or accidental ownership transfers can result in loss of administrative control

Likelihood

Low

Recommendation

Replace OwnableUpgradeable with Ownable2StepUpgradeable to enforce a two-phase ownership transfer that requires the new owner to explicitly accept ownership



RewardsDistributor.sweep() can remove tokens still owed to users

Resolved

Path

Delegation.sol / RewardsDistributor.sol

Function Name

`RewardsDistributor.sol`

Description

Rewards can remain payable for a token even after it is removed from `_rewardTokens`.

Users may still have `unclaimed[tokenId][t]` balances and/or rewards payable based on `accRPS` vs `rewardDebt`.

The `harvest()` function does not check `isRewardToken[t]` before paying rewards and can still pay historical reward tokens if a balance exists. However, `sweep()` only blocks tokens that are currently marked as reward tokens.

If the owner removes a reward token (`isRewardToken[token] = false`), `sweep()` becomes callable for that token even though users may still be owed rewards, allowing those liabilities to be swept.

Impact

Medium

Likelihood

Medium

Recommendation

Prevent sweeping any token that has ever been used as a reward token. The sweep guard should block tokens with outstanding or historical reward liabilities. If a true rescue mechanism is required, consider introducing a timelock, governance process, or a design that only allows sweeping once liabilities are proven to be zero.



Upgradeable init bug: `_nextTokenId = 1` won't apply to proxy storage

Resolved

Path

Delegation.sol

Function Name

`initialize()`

Description

In a UUPS proxy deployment, state variable initializers do not run in the proxy's storage, so `_nextTokenId` will start at 0 unless it is explicitly set in `initialize()`.

This means the first minted position will likely have `tokenId = 0`. While not inherently unsafe, this is often unexpected and can break off-chain assumptions.

Impact

Low

Likelihood

Low

Recommendation

Explicitly set `_nextTokenId = 1` inside `initialize()`.



Functional Tests

Some of the tests performed are mentioned below:

Delegation.sol

- ✓ Deposits correctly transfer underlying tokens, mint shares, and create new position NFTs.
- ✓ Delegated deposits properly assign operator and vault details to each new position.
- ✓ Positions created through delegated deposits enforce the lock duration and reject early withdrawals.
- ✓ Unlocked positions allow valid withdrawal actions without inconsistencies.
- ✓ Withdrawal requests correctly reduce active shares and move them into the pending state.
- ✓ Epoch identifiers reflect accurate vault state during withdrawal initiation.
- ✓ Claiming matured withdrawals returns underlying funds and clears pending state as expected.
- ✓ Share changes produced by vault interactions propagate correctly to the rewards system.

RewardsDistributor.sol

- ✓ Reward notifications allocate rewards proportionally based on active shares.
- ✓ Buffered rewards are handled correctly when shares are zero and flushed when needed.
- ✓ Reward accounting reflects accurate per-operator and per-token accumulation.
- ✓ onSharesIncrease accurately updates pool totals, reward debts, and unclaimed credit.
- ✓ onSharesDecrease properly recalculates reward debts and adjusts pool totals.
- ✓ Users can harvest individual or all rewards, receiving accurate accrued amounts.
- ✓ Reward debts and unclaimed balances correctly adjust after each harvest.
- ✓ Harvesting is allowed during paused states, ensuring user fund access at all times.

YODL.sol

- ✓ Minting to entity wallets enforces per-entity caps and global supply caps.



- ✓ `mintAndDelegate` correctly consumes entity quota, mints to the contract, approves delegation, and delegates through `Delegation.sol`.
- ✓ Quotas decrease accurately with each mint and revert when exceeding limits.
- ✓ Transfers respect pause state and revert when the token is globally paused.
- ✓ Transfers involving blacklisted accounts revert consistently.
- ✓ Minting and burning correctly update `totalSupply` while respecting the `ERC20Capped` limit.
- ✓ Blacklisted addresses cannot send or receive tokens.
- ✓ Only accounts with `BLACKLISTER_ROLE` can update blacklist state.
- ✓ Unblacklisting restores normal transfer behavior.
- ✓ `ERC20Permit` signatures validate correctly and update nonces as expected.

IOOCrowdsale.sol

- ✓ `latestRoundData` reads revert on stale, invalid, or incomplete rounds.
- ✓ Stablecoin feeds must remain within provided USD bands.
- ✓ ETH/USD deviation breaker correctly enforces maximum deviation thresholds.
- ✓ Token decimals are fetched on-chain and applied correctly in conversion logic.
- ✓ Simulated purchase paths (`_simulateLiveTotals`) calculate tokens and USD usage without modifying state.
- ✓ Simulated purchase paths (`_simulateLiveTotals`) calculate tokens and USD usage without modifying state.
- ✓ Live application (`_applyLiveAndCollectSegments`) updates tranche totals, buyer totals, and global sale totals precisely.
- ✓ Tranche transitions occur correctly when a tranche is fully consumed.
- ✓ Per-EOA cap, hard cap, and tranche caps are enforced reliably.
- ✓ `minTokenPurchase` prevents micro-buys below the configured threshold.
- ✓ `minTokensOut` ensures buyers receive at least the expected number of tokens.
- ✓ Finalized purchases mint YODL tokens and delegate them via `Delegation.sol`.
- ✓ Allowance workflows between the Crowdsale and Delegation contracts behave consistently.
- ✓ Minting and event emission match exact price-segmented allocations.
- ✓ Excess ETH is refunded to the buyer with correct remainder logic.



- ✓ `quoteBuyWithEth` and `quoteBuyWithStable` return accurate simulation values without modifying storage.
- ✓ Quotes reflect tranche transitions, cap limits, and token purchase constraints.

Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.

Threat Model

Contract	Function	Threats
Delegation.sol	depositAndDelegate()	Vault can front-run deposit and return manipulated shares to reduce user value. USDT-style non-standard approval flow exposes allowance race exploitation on deposit. No slippage guard lets operators force unfavorable share-minting conditions. RewardsIncrease can be front-run, letting attacker capture reward accrual before user joins.
Delegation.sol	_mintPosition()	Timestamp-based lock may overflow or be bypassed, enabling early withdrawals. TokenId increment can race on upgrade logic, causing overwritten or reused receipts.
Delegation.sol	onSharesIncrease()	Flash-loan share inflations capture large portions of buffered rewards. Baseline rewardDebt updated at join can create unfair retroactive reward assignment.
Delegation.sol	requestWithdraw()	Attacker can front-run onSharesDecrease to steal last-block reward before user. If multiple withdrawals hit same epoch, epochShares overwrites previous entries. Dust rounding in shares allows small but repeated value extraction.



Contract	Function	Threats
Delegation.sol	<code>_addPendingEpoch()</code>	Duplicate epoch push overwrites share amounts, creating incorrect pending state. Large epoch arrays cause gas griefing for withdraw/claim calls.
Delegation.sol	<code>claimWithdraw()</code>	Replaying or reordering epochIds can desync epochIndex and leave funds locked. Vault may return manipulated claim amounts, reducing expected underlying. Missing per-epoch checking may allow forcing invalid claim flows.
Delegation.sol	<code>_removePendingEpoch()</code>	Swap-and-pop reorders epochs, enabling unexpected behavior for multi-epoch claims. Bad index sync can permanently orphan pending epochs.
Delegation.sol	<code>initiateRedelegation()</code>	Redelegation allows bypassing lockDuration by instantly moving to pending state. Wholesale share movement enables reward-timing attacks for last-second gains. Operator vault reconfiguration can be front-run to siphon redelegated underlying.

Contract	Function	Threats
Delegation.sol	finalizeRedelegation()	Deposit into malicious new vault drains all redelegated funds. No slippage protection allows receiving far fewer shares than expected. EpochIds selection lets attacker finalize partial states, locking remainder.
Delegation.sol	cancelRedelegation()	Cancel can be front-run to manipulate returned share count. Rounding differences allow attacker to regain more shares than removed. Cancel path bypasses cliff, enabling early liquidity extraction.
RewardsDistributor.sol	harvest()	ERC20 callback reentrancy during transfer allows repeated harvests. Operator resolved from delegation may be zero, causing misrouted rewards.
RewardsDistributor.sol	_harvestToCredit()	Incorrect rewardDebt update lets attacker accumulate inflated unclaimed rewards. accRPS multiplication risks rounding exploits during share changes.
IOOCrowdsale.sol	buyWithEth()	Oracle update can be front-run to force worse ETH/USD price. Deviation breaker may reject correct price, causing forced revert attacks. Tranche crossing is manipulable—attackers buy tiny amount to push user upward.

Contract	Function	Threats
IOOCrowdsale.sol	_snapEthPrice()	Refund mechanism can be abused for gas grief or call reentrancy attempts. Price-snap-based USD conversions may shift during heavy MEV blocks. Malicious or stale feed data misprices token purchases. lastEthPrice1e18 can be manipulated to block future buys.
IOOCrowdsale.sol	_usdFromAmountETHWithSnap()	Rounding in ETH to USD conversion leaks small value repeatedly.
IOOCrowdsale.sol	_simulateLiveTotals()	Attacker can pre-fill current tranche to force higher-priced tranche for user. Buyer cap calculation may be bypassed using smart contract wallets. Rounding on Math.mulDiv allows minor dust leak attack.
IOOCrowdsale.sol	_applyLiveAndCollectSegments()	Race condition with other buys can cause over-paying for later segments. Multi-segment purchases can be MEV'ed to worsen user's effective price. Using dynamic sold state opens chance for partial state desync.
IOOCrowdsale.sol	_finalizeMintAndEmit()	Approve to Delegation may be front-run, allowing unauthorized pull. Math.mulDiv floor causes under-allocation of tokens in early segments. Event structure leaks tranche allocation that MEV bots can target.



Contract	Function	Threats
I00Crowdsale.sol	buyWithStable()	Stablecoins with transfer-fee or non-standard behavior break amount math. Oracle feed for stables can be manipulated even slightly to bypass sanity band. Exact pull via safeTransferFrom can revert if token is non-standard (USDT). USD conversions may round in attacker's favor during repeated buys.
I00Crowdsale.sol	_snapStablePrice()	Slight oracle deviation can cause stable to be mispriced outside band. Stale data allows attacker to buy tokens at unreasonably cheap satellite prices.
I00Crowdsale.sol	_usdFromAmountERC20WithSnap()	Decimal mismatch can inflate USD value of attacker's input amount.
I00Crowdsale.sol	quoteBuyWithEth(), quoteBuyWithStable(), quoteByUsd()	Attacker uses quotes to probe tranche state for MEV timing. Stale feed used in quote gives misleading results for user decisions. Simulation front-run: attacker sees user quote then fills tranche before user.

Closing Summary

In this report, we have considered the security of YODL Smart Contracts. We performed our audit according to the procedure described above.

Critical, High, Medium, Low and Informational Issues found during the Audit, Yodl team acknowledged few and resolved the remaining issues

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.



About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers. With seven years of expertise, we've secured over 1500 projects globally, averting over \$3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.



8+ Years of Expertise	1M+ Lines of Code Audited
50+ Chains Supported	1500+ Projects Secured

Follow Our Journey



AUDIT REPORT

December 2025

For



Yodl



Canada, India, Singapore, UAE, UK

www.quillaudits.com

audits@quillaudits.com